

EXP-3	Build and train a CNN model for Face recognition
-------	--

AIM:

- To build and train a deep CNN model to recognize faces from the Labeled Faces in the Wild (LFW) dataset.
- To evaluate the performance of the CNN model on unseen facial images.

Algorithm:

1. **Import Libraries:** Import required libraries like TensorFlow, Keras, sklearn, numpy, etc.
2. **Load Dataset:** Use `fetch_lfw_people()` from sklearn to load the LFW dataset with a minimum number of faces per person.
3. **Preprocess Data:**
 - Normalize images to range [0, 1].
 - Encode target labels using `LabelEncoder`.
 - Split data into training and testing sets using `train_test_split`.
4. **Reshape Input:** Reshape flattened images into 4D tensors with channel dimension suitable for CNN.
5. **Augment Data:** Use `ImageDataGenerator` to apply transformations like flipping, rotation, zooming, etc.
6. **Build CNN Model:**
 - Use multiple convolutional layers with ReLU and Batch Normalization.
 - Add MaxPooling for downsampling.
 - Flatten and pass through Dense layers with Dropout for regularization.
 - Use a softmax output layer for multi-class classification.

7. **Compile Model:** Use Adam optimizer and sparse categorical crossentropy loss.
8. **Train Model:** Fit the model on training data and validate on the test set.
9. **Evaluate Model:** Compute accuracy on test data and print the final accuracy.

Code:

```
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import sklearn.datasets

# Load LFW dataset
lfw_people = sklearn.datasets.fetch_lfw_people(min_faces_per_person=70, resize=0.4)
X = lfw_people.data
y = lfw_people.target
target_names = lfw_people.target_names

# Normalize the images
X = X.astype('float32') / 255.0

# Encode the labels
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(y)

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Reshape images to 4D tensors
image_width = 50
X_train = X_train.reshape(X_train.shape[0], image_width, 37, 1)
X_test = X_test.reshape(X_test.shape[0], image_width, 37, 1)

# Data augmentation
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=30,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
```

```

        zoom_range=0.2,
        horizontal_flip=True
    )

train_generator = train_datagen.flow(X_train, y_train, batch_size=32)

# Define CNN model
def build_deeper_face_recognition_model(input_shape=(50, 37, 1), num_classes=7):
    model = models.Sequential()
    model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=input_shape, padding='same'))
    model.add(layers.BatchNormalization())
    model.add(layers.MaxPooling2D((2, 2)))

    model.add(layers.Conv2D(64, (3, 3), activation='relu', padding='same'))
    model.add(layers.BatchNormalization())
    model.add(layers.MaxPooling2D((2, 2)))

    model.add(layers.Conv2D(128, (3, 3), activation='relu', padding='same'))
    model.add(layers.BatchNormalization())
    model.add(layers.MaxPooling2D((2, 2)))

    model.add(layers.Conv2D(256, (3, 3), activation='relu', padding='same'))
    model.add(layers.BatchNormalization())
    model.add(layers.MaxPooling2D((1, 1)))

    model.add(layers.Flatten())
    model.add(layers.Dense(1024, activation='relu'))
    model.add(layers.Dropout(0.5))
    model.add(layers.Dense(512, activation='relu'))
    model.add(layers.Dropout(0.5))
    model.add(layers.Dense(num_classes, activation='softmax'))

    model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
    return model

# Build and train model
model = build_deeper_face_recognition_model(input_shape=(50, 37, 1),
num_classes=len(lfw_people.target_names))
history = model.fit(train_generator, epochs=20, validation_data=(X_test, y_test))

# Evaluate the model
test_loss, test_acc = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {test_acc:.4f}")

```

Output:



Result:

A deep convolutional neural network was successfully built and trained for multi-class face recognition. The model achieved a good accuracy on unseen face images from the LFW dataset.

